



소민님 보세요!!!! fastapi 에서 azure db 연결하기

FastAPI와 Azure PostgreSQL을 연결하는 방법을 단계별로 알려드리겠습니다!

FastAPI + Azure PostgreSQL 연결 가이드

STEP 1: 프로젝트 구조 만들기

폴더 구조

```
carecircle-backend/  
├── app/  
│   ├── __init__.py  
│   ├── main.py          # FastAPI 앱  
│   ├── database.py      # DB 연결 설정  
│   ├── models.py        # SQLAlchemy 모델  
│   ├── schemas.py       # Pydantic 스키마  
│   ├── crud.py          # CRUD 작업  
│   └── routers/         # API 라우터  
│       ├── __init__.py  
│       ├── users.py  
│       ├── patients.py  
│       └── caregivers.py  
├── tests/  
│   └── test_api.py  
├── .env                 # 환경 변수  
├── requirements.txt     # 의존성  
└── README.md
```

STEP 2: 필요한 라이브러리 설치

requirements.txt 생성

```
# FastAPI 핵심
fastapi==0.104.1
uvicorn[standard]==0.24.0

# 데이터베이스
sqlalchemy==2.0.23
psycopg2-binary==2.9.9
asyncpg==0.29.0

# 환경 변수
python-dotenv==1.0.0

# 데이터 검증
pydantic==2.5.0
pydantic-settings==2.1.0

# 인증 (나중에 사용)
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
python-multipart==0.0.6

# Azure 관련
azure-identity==1.15.0
azure-keyvault-secrets==4.7.0

# 유틸리티
python-dateutil==2.8.2
```

설치

```
# 가상환경 생성 (권장)
python -m venv .venv
```

```
# 가상환경 활성화
# Windows
.venv\Scripts\activate
# Mac/Linux
source .venv/bin/activate

# 라이브러리 설치
pip install -r requirements.txt
```

STEP 3: 환경 변수 설정

.env 파일 생성

```
# Azure PostgreSQL 연결 정보
DATABASE_HOST=neulbomcare-db.postgres.database.azure.com
DATABASE_PORT=5432
DATABASE_NAME=neulbom
DATABASE_USER=adminuser
DATABASE_PASSWORD=****

# DATABASE_URL (전체 연결 문자열)
DATABASE_URL=postgresql://adminuser:****@neulbomcare-db.postgres.d
atabase.azure.com:5432/neulbom?sslmode=require

#이건 올바르게 않습니다!! 소민님 설정대로 하세요!!!
# FastAPI 설정
APP_NAME=CareCircle API
DEBUG=True
API_VERSION=v1

# JWT 시크릿 (나중에 사용)
SECRET_KEY=*****
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30

# Kakao OAuth
```

```
KAKAO_REST_API_KEY=*****
KAKAO_REDIRECT_URI=http://localhost:8000/auth/kakao/callback
KAKAO_CLIENT_SECRET=your_kakao_client_secret_here_if_needed

# CORS (Frontend URL)
FRONTEND_URL=http://localhost:3000
```

⚠️ .gitignore에 추가

```
# .gitignore
.env
venv/
__pycache__/
*.pyc
.pytest_cache/
```

🔌 STEP 4: 데이터베이스 연결 설정

app/database.py

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from dotenv import load_dotenv
import os

# 환경 변수 로드
load_dotenv()

# 데이터베이스 URL
DATABASE_URL = os.getenv("DATABASE_URL")

# SQLAlchemy 엔진 생성
engine = create_engine(
    DATABASE_URL,
    pool_pre_ping=True, # 연결 확인
```

```

pool_size=10,      # 연결 풀 크기
max_overflow=20,   # 추가 연결 허용
echo=True          # SQL 로그 출력 (개발 시)
)

# 세션 생성
SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

# Base 클래스 (모든 모델의 부모)
Base = declarative_base()

# 의존성 주입용 함수
def get_db():
    """
    데이터베이스 세션을 생성하고 반환합니다.
    FastAPI의 Depends에서 사용됩니다.
    """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

STEP 5: SQLAlchemy 모델 정의

app/models.py (일부 예시)

```

from sqlalchemy import (
    Column, String, Integer, BigInteger, Boolean,
    DateTime, Date, Float, Enum, ARRAY, Text, ForeignKey
)
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.orm import relationship

```

```

from datetime import datetime
import enum

from .database import Base

# =====
# Enum 타입 정의
# =====

class UserTypeEnum(str, enum.Enum):
    GUARDIAN = "guardian"
    CAREGIVER = "caregiver"

class GenderEnum(str, enum.Enum):
    MALE = "Male"
    FEMALE = "Female"

class CareLevelEnum(str, enum.Enum):
    LEVEL_1 = "1등급"
    LEVEL_2 = "2등급"
    LEVEL_3 = "3등급"
    LEVEL_4 = "4등급"
    LEVEL_5 = "5등급"
    COGNITIVE = "인지지원등급"
    NONE = "등급외"

# =====
# 모델 정의
# =====

class User(Base):
    """사용자 모델 (보호자 + 간병인)"""
    __tablename__ = "users"

    user_id = Column(BigInteger, primary_key=True, index=True)
    email = Column(String(100), unique=True, nullable=True, index=True)
    password_hash = Column(String(255), nullable=True)
    name = Column(String(50), nullable=False)

```

```

    phone_number = Column(String(20), unique=True, nullable=True, index=True)
    user_type = Column(Enum(UserTypeEnum), nullable=False)
    profile_image_url = Column(Text, nullable=True)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # 관계 설정
    guardian = relationship("Guardian", back_populates="user", uselist=False)
    caregiver = relationship("Caregiver", back_populates="user", uselist=False)
    personality_tests = relationship("PersonalityTest", back_populates="user")
    reviews = relationship("Review", back_populates="reviewer")

class Guardian(Base):
    """보호자 모델"""
    __tablename__ = "guardians"

    guardian_id = Column(BigInteger, primary_key=True, index=True)
    user_id = Column(BigInteger, ForeignKey("users.user_id", ondelete="CASCADE"), unique=True, nullable=False)
    address = Column(String(255), nullable=True)
    relationship_to_patient = Column(String(50), nullable=True)
    emergency_contact = Column(String(20), nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # 관계
    user = relationship("User", back_populates="guardian")
    patients = relationship("Patient", back_populates="guardian")

```

```

class Patient(Base):
    """환자 모델"""
    __tablename__ = "patients"

    patient_id = Column(BigInteger, primary_key=True, index=True)
    guardian_id = Column(BigInteger, ForeignKey("guardians.guardian_id", ondelete="CASCADE"), nullable=False)
    name = Column(String(50), nullable=False)
    birth_date = Column(Date, nullable=False)
    gender = Column(Enum(GenderEnum), nullable=False)
    height = Column(Integer, nullable=True)
    weight = Column(Integer, nullable=True)
    care_address = Column(String(255), nullable=False)
    region_code = Column(String(50), nullable=False, index=True)
    care_level = Column(Enum(CareLevelEnum), nullable=True)
    profile_image_url = Column(Text, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # 관계
    guardian = relationship("Guardian", back_populates="patients")
    health_conditions = relationship("HealthCondition", back_populates="patient")
    medications = relationship("Medication", back_populates="patient")
    dietary_preferences = relationship("DietaryPreference", back_populates="patient")
    personality = relationship("PatientPersonality", back_populates="patient", uselist=False)
    schedules = relationship("Schedule", back_populates="patient")
    meal_plans = relationship("MealPlan", back_populates="patient")

class Caregiver(Base):
    """간병인 모델"""
    __tablename__ = "caregivers"

    caregiver_id = Column(BigInteger, primary_key=True, index=True)

```



```

    user_id = Column(BigInteger, ForeignKey("users.user_id", ondelete="CASCADE"), unique=True, nullable=False)
    experience_years = Column(Integer, default=0)
    certifications = Column(String(255), nullable=True)
    specialties = Column(ARRAY(Text), nullable=True)
    service_region = Column(String(50), nullable=True, index=True)
    has_vehicle = Column(Boolean, default=False)
    hourly_rate = Column(Integer, nullable=True)
    avg_rating = Column(Float, default=0.0)
    total_reviews = Column(Integer, default=0)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

```

```

# 관계
user = relationship("User", back_populates="caregiver")
personality = relationship("CaregiverPersonality", back_populates="caregiver", uselist=False)
matching_results = relationship("MatchingResult", back_populates="caregiver")

```

```

class HealthCondition(Base):
    """질병 정보 모델"""
    __tablename__ = "health_conditions"

    condition_id = Column(BigInteger, primary_key=True, index=True)
    patient_id = Column(BigInteger, ForeignKey("patients.patient_id", ondelete="CASCADE"), nullable=False)
    disease_name = Column(String(100), nullable=False)
    note = Column(Text, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

    # 관계
    patient = relationship("Patient", back_populates="health_conditions")

```

```

class Medication(Base):

```

```

"""약 정보 모델"""
__tablename__ = "medications"

med_id = Column(BigInteger, primary_key=True, index=True)
patient_id = Column(BigInteger, ForeignKey("patients.patient_id", ondelete="CASCADE"), nullable=False)
name = Column(String(100), nullable=False)
dosage = Column(String(50), nullable=True)
frequency = Column(String(50), nullable=True)
intake_method = Column(String(100), nullable=True)
image_url = Column(Text, nullable=True)
created_at = Column(DateTime, default=datetime.utcnow)

# 관계
patient = relationship("Patient", back_populates="medications")

class DietaryPreference(Base):
    """식사 정보 모델 (수정됨)"""
    __tablename__ = "dietary_preferences"

    diet_id = Column(BigInteger, primary_key=True, index=True)
    patient_id = Column(BigInteger, ForeignKey("patients.patient_id", ondelete="CASCADE"), nullable=False)
    allergy_foods = Column.ARRAY(Text), nullable=True)
    restriction_foods = Column.ARRAY(Text), nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

    # 관계
    patient = relationship("Patient", back_populates="dietary_preferences")

class PersonalityTest(Base):
    """성향 테스트 모델"""
    __tablename__ = "personality_tests"

    test_id = Column(BigInteger, primary_key=True, index=True)
    user_id = Column(BigInteger, ForeignKey("users.user_id", ondelete="CA

```

```

SCADE"), nullable=False)
    empathy_score = Column(Float, default=0)
    activity_score = Column(Float, default=0)
    patience_score = Column(Float, default=0)
    independence_score = Column(Float, default=0)
    raw_test_answers = Column(JSONB, nullable=True)
    ai_analysis_text = Column(Text, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

# 관계
user = relationship("User", back_populates="personality_tests")

class PatientPersonality(Base):
    """환자 성향 모델"""
    __tablename__ = "patient_personality"

    personality_id = Column(BigInteger, primary_key=True, index=True)
    patient_id = Column(BigInteger, ForeignKey("patients.patient_id", ondelete="CASCADE"), unique=True, nullable=False)
    empathy_score = Column(Float, default=0)
    activity_score = Column(Float, default=0)
    patience_score = Column(Float, default=0)
    independence_score = Column(Float, default=0)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

# 관계
patient = relationship("Patient", back_populates="personality")

class CaregiverPersonality(Base):
    """간병인 성향 모델"""
    __tablename__ = "caregiver_personality"

    personality_id = Column(BigInteger, primary_key=True, index=True)
    caregiver_id = Column(BigInteger, ForeignKey("caregivers.caregiver_id", ondelete="CASCADE"), unique=True, nullable=False)

```

```

empathy_score = Column(Float, default=0)
activity_score = Column(Float, default=0)
patience_score = Column(Float, default=0)
independence_score = Column(Float, default=0)
updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

```

```

# 관계

```

```

caregiver = relationship("Caregiver", back_populates="personality")

```

```

class MatchingResult(Base):

```

```

    """매칭 결과 모델"""

```

```

    __tablename__ = "matching_results"

```

```

    matching_id = Column(BigInteger, primary_key=True, index=True)

```

```

    request_id = Column(BigInteger, ForeignKey("matching_requests.request_id", ondelete="CASCADE"), nullable=False)

```

```

    caregiver_id = Column(BigInteger, ForeignKey("caregivers.caregiver_id", ondelete="CASCADE"), nullable=False)

```

```

    total_score = Column(Float, nullable=False)

```

```

    grade = Column(String(3), nullable=False)

```

```

    ai_comment = Column(Text, nullable=True)

```

```

    status = Column(String(20), default="recommended")

```

```

    contract_start_date = Column(Date, nullable=True)

```

```

    contract_end_date = Column(Date, nullable=True)

```

```

    created_at = Column(DateTime, default=datetime.utcnow)

```

```

    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

```

```

# 관계

```

```

caregiver = relationship("Caregiver", back_populates="matching_results")

```

```

class Schedule(Base):

```

```

    """케어 일정 모델"""

```

```

    __tablename__ = "schedules"

```

```

    schedule_id = Column(BigInteger, primary_key=True, index=True)
    patient_id = Column(BigInteger, ForeignKey("patients.patient_id", ondelete="CASCADE"), nullable=False)
    matching_id = Column(BigInteger, ForeignKey("matching_results.matching_id", ondelete="CASCADE"), nullable=True)
    care_date = Column(Date, nullable=False, index=True)
    is_ai_generated = Column(Boolean, default=True)
    status = Column(String(20), default="scheduled")
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

```

```

# 관계

```

```

patient = relationship("Patient", back_populates="schedules")

```

```

class MealPlan(Base):

```

```

    """추천 식단 모델"""

```

```

    __tablename__ = "meal_plans"

```

```

    plan_id = Column(BigInteger, primary_key=True, index=True)
    patient_id = Column(BigInteger, ForeignKey("patients.patient_id", ondelete="CASCADE"), nullable=False)
    meal_date = Column(Date, nullable=False, index=True)
    meal_type = Column(String(20), nullable=False)
    menu_name = Column(String(200), nullable=False)
    ingredients = Column(Text, nullable=True)
    nutrition_info = Column(JSONB, nullable=True)
    cooking_tips = Column(Text, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

```

```

# 관계

```

```

patient = relationship("Patient", back_populates="meal_plans")

```

```

class Review(Base):

```

```

    """리뷰 모델"""

```

```

__tablename__ = "reviews"

review_id = Column(BigInteger, primary_key=True, index=True)
matching_id = Column(BigInteger, ForeignKey("matching_results.matchin
g_id", ondelete="CASCADE"), nullable=False)
reviewer_id = Column(BigInteger, ForeignKey("users.user_id", ondelete
="CASCADE"), nullable=False)
reviewer_type = Column(Enum(UserTypeEnum), nullable=False)
rating = Column(Integer, nullable=False)
comment = Column(Text, nullable=True)
created_at = Column(DateTime, default=datetime.utcnow)

# 관계
reviewer = relationship("User", back_populates="reviews")

# 나머지 모델들도 동일한 패턴으로 작성...

```



STEP 6: Pydantic 스키마 정의

app/schemas.py (일부 예시)

```

from pydantic import BaseModel, EmailStr, Field, validator
from typing import Optional, List
from datetime import date, datetime
from enum import Enum

# =====
# Enum 정의
# =====

class UserType(str, Enum):
    GUARDIAN = "guardian"
    CAREGIVER = "caregiver"

class Gender(str, Enum):
    MALE = "Male"
    FEMALE = "Female"

```

```

# =====
# User 스키마
# =====

class UserBase(BaseModel):
    """User 기본 스키마"""
    email: Optional[EmailStr] = None
    name: str = Field(..., min_length=1, max_length=50)
    phone_number: Optional[str] = Field(None, pattern=r"^010-\d{4}-\d{4}$")
    user_type: UserType

class UserCreate(UserBase):
    """User 생성 스키마"""
    password: str = Field(..., min_length=8)

class UserUpdate(BaseModel):
    """User 수정 스키마"""
    email: Optional[EmailStr] = None
    name: Optional[str] = Field(None, min_length=1, max_length=50)
    phone_number: Optional[str] = None
    profile_image_url: Optional[str] = None

class UserResponse(UserBase):
    """User 응답 스키마"""
    user_id: int
    profile_image_url: Optional[str] = None
    is_active: bool
    created_at: datetime
    updated_at: datetime

class Config:
    from_attributes = True # Pydantic v2 (이전 orm_mode)

# =====
# Guardian 스키마

```

```
# =====

class GuardianBase(BaseModel):
    """Guardian 기본 스키마"""
    address: Optional[str] = None
    relationship_to_patient: Optional[str] = None
    emergency_contact: Optional[str] = None

class GuardianCreate(GuardianBase):
    """Guardian 생성 스키마"""
    user_id: int

class GuardianResponse(GuardianBase):
    """Guardian 응답 스키마"""
    guardian_id: int
    user_id: int
    created_at: datetime
    updated_at: datetime

class Config:
    from_attributes = True

# =====
# Patient 스키마
# =====

class PatientBase(BaseModel):
    """Patient 기본 스키마"""
    name: str = Field(..., min_length=1, max_length=50)
    birth_date: date
    gender: Gender
    height: Optional[int] = Field(None, gt=0, lt=300)
    weight: Optional[int] = Field(None, gt=0, lt=500)
    care_address: str
    region_code: str
    care_level: Optional[str] = None
```



```

class PatientCreate(PatientBase):
    """Patient 생성 스키마"""
    guardian_id: int

class PatientUpdate(BaseModel):
    """Patient 수정 스키마"""
    name: Optional[str] = None
    height: Optional[int] = None
    weight: Optional[int] = None
    care_address: Optional[str] = None
    profile_image_url: Optional[str] = None

class PatientResponse(PatientBase):
    """Patient 응답 스키마"""
    patient_id: int
    guardian_id: int
    profile_image_url: Optional[str] = None
    created_at: datetime
    updated_at: datetime

class Config:
    from_attributes = True

# =====
# Caregiver 스키마
# =====

class CaregiverBase(BaseModel):
    """Caregiver 기본 스키마"""
    experience_years: int = Field(default=0, ge=0)
    certifications: Optional[str] = None
    specialties: Optional[List[str]] = None
    service_region: Optional[str] = None
    has_vehicle: bool = False
    hourly_rate: Optional[int] = Field(None, gt=0, lt=1000000)

class CaregiverCreate(CaregiverBase):

```

```

"""Caregiver 생성 스키마"""
user_id: int

class CaregiverUpdate(BaseModel):
    """Caregiver 수정 스키마"""
    experience_years: Optional[int] = None
    certifications: Optional[str] = None
    specialties: Optional[List[str]] = None
    hourly_rate: Optional[int] = None

class CaregiverResponse(CaregiverBase):
    """Caregiver 응답 스키마"""
    caregiver_id: int
    user_id: int
    avg_rating: float
    total_reviews: int
    created_at: datetime
    updated_at: datetime

class Config:
    from_attributes = True

# =====
# 기타 응답 스키마
# =====

class MessageResponse(BaseModel):
    """메시지 응답"""
    message: str

class ErrorResponse(BaseModel):
    """에러 응답"""
    detail: str

# 나머지 스키마들도 동일한 패턴으로 작성...

```



STEP 7: CRUD 작업 구현

app/crud.py (일부 예시)

```
from sqlalchemy.orm import Session
from sqlalchemy import and_, or_
from typing import List, Optional
from datetime import date

from . import models, schemas

# =====
# User CRUD
# =====

def get_user(db: Session, user_id: int):
    """사용자 조회"""
    return db.query(models.User).filter(models.User.user_id == user_id).first()

def get_user_by_email(db: Session, email: str):
    """이메일로 사용자 조회"""
    return db.query(models.User).filter(models.User.email == email).first()

def get_users(db: Session, skip: int = 0, limit: int = 100):
    """사용자 목록 조회"""
    return db.query(models.User).offset(skip).limit(limit).all()

def create_user(db: Session, user: schemas.UserCreate):
    """사용자 생성"""
    # 비밀번호 해싱은 나중에 추가
    db_user = models.User(
        email=user.email,
        password_hash=user.password, # TODO: 해싱 필요
        name=user.name,
        phone_number=user.phone_number,
        user_type=user.user_type
    )
```

```

    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user

def update_user(db: Session, user_id: int, user: schemas.UserUpdate):
    """사용자 수정"""
    db_user = get_user(db, user_id)
    if db_user:
        update_data = user.dict(exclude_unset=True)
        for key, value in update_data.items():
            setattr(db_user, key, value)
        db.commit()
        db.refresh(db_user)
    return db_user

def delete_user(db: Session, user_id: int):
    """사용자 삭제"""
    db_user = get_user(db, user_id)
    if db_user:
        db.delete(db_user)
        db.commit()
    return db_user

# =====
# Patient CRUD
# =====

def get_patient(db: Session, patient_id: int):
    """환자 조회"""
    return db.query(models.Patient).filter(models.Patient.patient_id == patient_id).first()

def get_patients_by_guardian(db: Session, guardian_id: int):
    """보호자의 환자 목록"""
    return db.query(models.Patient).filter(models.Patient.guardian_id == guardian_id).all()

```

```

def create_patient(db: Session, patient: schemas.PatientCreate):
    """환자 생성"""
    db_patient = models.Patient(**patient.dict())
    db.add(db_patient)
    db.commit()
    db.refresh(db_patient)
    return db_patient

def update_patient(db: Session, patient_id: int, patient: schemas.PatientUpdate):
    """환자 수정"""
    db_patient = get_patient(db, patient_id)
    if db_patient:
        update_data = patient.dict(exclude_unset=True)
        for key, value in update_data.items():
            setattr(db_patient, key, value)
        db.commit()
        db.refresh(db_patient)
    return db_patient

# =====
# Caregiver CRUD
# =====

def get_caregiver(db: Session, caregiver_id: int):
    """간병인 조회"""
    return db.query(models.Caregiver).filter(models.Caregiver.caregiver_id =
= caregiver_id).first()

def get_caregivers(
    db: Session,
    skip: int = 0,
    limit: int = 100,
    region: Optional[str] = None,
    min_rating: Optional[float] = None
):

```

```

"""간병인 목록 조회 (필터링)"""
query = db.query(models.Caregiver)

if region:
    query = query.filter(models.Caregiver.service_region == region)

if min_rating:
    query = query.filter(models.Caregiver.avg_rating >= min_rating)

return query.offset(skip).limit(limit).all()

def create_caregiver(db: Session, caregiver: schemas.CaregiverCreate):
    """간병인 생성"""
    db_caregiver = models.Caregiver(**caregiver.dict())
    db.add(db_caregiver)
    db.commit()
    db.refresh(db_caregiver)
    return db_caregiver

# =====
# 복합 쿼리 예시
# =====

def get_patient_with_health_info(db: Session, patient_id: int):
    """환자 + 건강 정보 조회"""
    patient = db.query(models.Patient).filter(
        models.Patient.patient_id == patient_id
    ).first()

    if patient:
        # 관련 데이터 로드 (eager loading)
        db.refresh(patient)
        # SQLAlchemy가 자동으로 relationship 로드

    return patient

def search_caregivers_by_specialty(db: Session, specialty: str):

```

```

"""전문 분야로 간병인 검색"""
return db.query(models.Caregiver).filter(
    models.Caregiver.specialties.contains([specialty])
).all()

# 나머지 CRUD 작업들...

```

STEP 8: FastAPI 라우터 생성

app/routers/users.py

```

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from typing import List

from .. import crud, schemas, models
from ..database import get_db

router = APIRouter(
    prefix="/users",
    tags=["users"]
)

@router.post("/", response_model=schemas.UserResponse, status_code=status.HTTP_201_CREATED)
def create_user(user: schemas.UserCreate, db: Session = Depends(get_db)):
    """
    새로운 사용자 생성

    - **email**: 이메일 (선택)
    - **name**: 이름 (필수)
    - **phone_number**: 전화번호 (선택)
    - **user_type**: 사용자 유형 (guardian 또는 caregiver)
    - **password**: 비밀번호 (필수, 8자 이상)
    """
    # 이메일 중복 체크

```

```

if user.email:
    db_user = crud.get_user_by_email(db, email=user.email)
    if db_user:
        raise HTTPException(
            status_code=400,
            detail="이미 등록된 이메일입니다."
        )

    return crud.create_user(db=db, user=user)

@router.get("/{user_id}", response_model=schemas.UserResponse)
def read_user(user_id: int, db: Session = Depends(get_db)):
    """사용자 조회"""
    db_user = crud.get_user(db, user_id=user_id)
    if db_user is None:
        raise HTTPException(status_code=404, detail="사용자를 찾을 수 없습니다.")
    return db_user

@router.get("/", response_model=List[schemas.UserResponse])
def read_users(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    """사용자 목록 조회"""
    users = crud.get_users(db, skip=skip, limit=limit)
    return users

@router.put("/{user_id}", response_model=schemas.UserResponse)
def update_user(
    user_id: int,
    user: schemas.UserUpdate,
    db: Session = Depends(get_db)
):
    """사용자 정보 수정"""
    db_user = crud.update_user(db, user_id=user_id, user=user)
    if db_user is None:
        raise HTTPException(status_code=404, detail="사용자를 찾을 수 없습니다.")
    return db_user

```



```

@router.delete("/{user_id}", response_model=schemas.MessageResponse)
def delete_user(user_id: int, db: Session = Depends(get_db)):
    """사용자 삭제"""
    db_user = crud.delete_user(db, user_id=user_id)
    if db_user is None:
        raise HTTPException(status_code=404, detail="사용자를 찾을 수 없습니다.")
    return {"message": "사용자가 삭제되었습니다."}

```

app/routers/patients.py

```

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from typing import List

from .. import crud, schemas
from ..database import get_db

router = APIRouter(
    prefix="/patients",
    tags=["patients"]
)

@router.post("/", response_model=schemas.PatientResponse, status_code=status.HTTP_201_CREATED)
def create_patient(patient: schemas.PatientCreate, db: Session = Depends(get_db)):
    """새로운 환자 생성"""
    return crud.create_patient(db=db, patient=patient)

@router.get("/{patient_id}", response_model=schemas.PatientResponse)
def read_patient(patient_id: int, db: Session = Depends(get_db)):
    """환자 조회"""
    db_patient = crud.get_patient(db, patient_id=patient_id)
    if db_patient is None:
        raise HTTPException(status_code=404, detail="환자를 찾을 수 없습니다.")

```

```

다.")
    return db_patient

@router.get("/guardian/{guardian_id}", response_model=List[schemas.PatientResponse])
def read_patients_by_guardian(guardian_id: int, db: Session = Depends(get_db)):
    """보호자의 환자 목록 조회"""
    patients = crud.get_patients_by_guardian(db, guardian_id=guardian_id)
    return patients

@router.put("/{patient_id}", response_model=schemas.PatientResponse)
def update_patient(
    patient_id: int,
    patient: schemas.PatientUpdate,
    db: Session = Depends(get_db)
):
    """환자 정보 수정"""
    db_patient = crud.update_patient(db, patient_id=patient_id, patient=patient)
    if db_patient is None:
        raise HTTPException(status_code=404, detail="환자를 찾을 수 없습니다.")
    return db_patient

```

STEP 9: 메인 앱 생성

app/main.py

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from dotenv import load_dotenv
import os

from .database import engine, Base
from .routers import users, patients

```

```

# 환경 변수 로드
load_dotenv()

# 데이터베이스 테이블 생성 (개발 시에만, 프로덕션에서는 마이그레이션 사용)
# Base.metadata.create_all(bind=engine)

# FastAPI 앱 생성
app = FastAPI(
    title=os.getenv("APP_NAME", "CareCircle API"),
    description="가족 간병 협업 플랫폼 API",
    version=os.getenv("API_VERSION", "v1"),
    docs_url="/docs",
    redoc_url="/redoc"
)

# CORS 설정
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # 프로덕션에서는 특정 도메인만 허용
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 라우터 등록
app.include_router(users.router, prefix="/api/v1")
app.include_router(patients.router, prefix="/api/v1")

# 루트 엔드포인트
@app.get("/")
def read_root():
    """API 상태 확인"""
    return {
        "message": "CareCircle API",
        "status": "running",
        "version": os.getenv("API_VERSION", "v1")
    }

```

```
# 헬스체크 엔드포인트
@app.get("/health")
def health_check():
    """헬스 체크"""
    return {"status": "healthy"}
```

STEP 10: 앱 실행

로컬에서 실행

```
# 개발 모드로 실행
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# 또는 더 자세한 로그
uvicorn app.main:app --reload --log-level debug
```

접속 확인

브라우저에서:

- API 문서: <http://localhost:8000/docs>
- ReDoc: <http://localhost:8000/redoc>
- API 상태: <http://localhost:8000/>

STEP 11: API 테스트

Swagger UI에서 테스트 (추천!)

1. <http://localhost:8000/docs> 접속
2. 원하는 엔드포인트 클릭
3. "Try it out" 버튼 클릭
4. 파라미터 입력
5. "Execute" 버튼 클릭
6. 응답 확인

cURL로 테스트

1. 사용자 생성

```
curl -X POST "http://localhost:8000/api/v1/users/" \
-H "Content-Type: application/json" \
-d '{
  "email": "test@example.com",
  "name": "테스트",
  "phone_number": "010-1234-5678",
  "user_type": "guardian",
  "password": "password123"
}'
```

2. 사용자 조회

```
curl -X GET "http://localhost:8000/api/v1/users/1"
```

3. 사용자 목록

```
curl -X GET "http://localhost:8000/api/v1/users/"
```

Python requests로 테스트

```
# test_api.py
```

```
import requests
```

```
BASE_URL = "http://localhost:8000/api/v1"
```

1. 사용자 생성

```
response = requests.post(
    f"{BASE_URL}/users/",
    json={
        "email": "test@example.com",
        "name": "테스트",
        "phone_number": "010-1234-5678",
        "user_type": "guardian",
        "password": "password123"
    }
)
print("생성:", response.json())
```

2. 사용자 조회

```
user_id = response.json()["user_id"]
response = requests.get(f"{BASE_URL}/users/{user_id}")
print("조회:", response.json())
```

3. 사용자 수정

```
response = requests.put(
    f"{BASE_URL}/users/{user_id}",
    json={"name": "수정된 이름"}
)
print("수정:", response.json())
```

실행 결과 예시

성공 시:

```
INFO:   Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:   Started reloader process [12345] using StatReload
INFO:   Started server process [12346]
INFO:   Waiting for application startup.
INFO:   Application startup complete.
```

API 응답 예시:

```
{
  "user_id": 1,
  "email": "test@example.com",
  "name": "테스트",
  "phone_number": "010-1234-5678",
  "user_type": "guardian",
  "profile_image_url": null,
  "is_active": true,
  "created_at": "2024-11-21T14:30:00",
  "updated_at": "2024-11-21T14:30:00"
}
```

다음 단계

- ✓ 1. FastAPI 기본 설정 완료
- ✓ 2. DB 연결 완료
- ✓ 3. 모델 정의 완료
- ✓ 4. CRUD 작업 완료
- ✓ 5. API 엔드포인트 완료

다음:

- 인증/인가 (JWT)
- 파일 업로드
- Azure AI 통합
- 프론트엔드 연동